Task No:- 8 Implement python generator and decorates

Date :- 17/9/25

Aim:-

8. write python program that includes a generator function to produce a sequence of numbers. The generator should be able To:

a. produce a sequence of numbers from when provided with start, end, and step values.

b. produce a default sequence of numbers starting from 0, ending at 10, and with a step of 1 if no values are provided, provide a sequence of numbers when provided with start, end and stepvalues.

1. Define Generator function:
   - Define the function number_sequence (start, end, step=1)
2. initialize current value:
   - set current to the value of start.
3. Generate sequence:
   - while current is less than or equal to end:
   - Yield the current value of current.
   - Increment current by step.
4. Get user Input:
   - Read the starting number (start) from user input.
   - Read the ending number (end) from user input.
   - Read the step value from user input.
5. Create Generator object:
   - Create a Generator object by calling number_sequence. (start, end, step) with user-provided values.
6. Print Generated sequence:
   - Iterate over the values produced by the generator object.
   - Print each value.

output:

```
Enter the starting Numbers: 1
Enter the ending Numbers: 50
Enter the step value:
1
6
11
16
21
26
31
36
41
46.
```

Program:

```
def number_sequence(start, end, step=1):
    current = start
    while current <= end:
        yield current
        current += step
start = int(input("enter the starting Numbers:"))
end = int(input("enter the ending Numbers:"))
step = int(input("enter the step value:"))
sequence_generator = number_sequence(start, end, step)
for numbers in sequence_generator:
    print(numbers).
```

Produce a default sequence of numbers strating from 0, ending at 10, and with a step of 1 if no values are provided.

Algorithm:

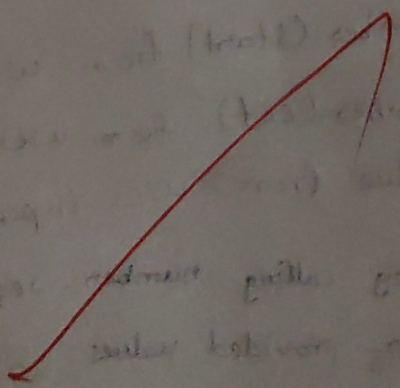
1. start function:
   - Define the function my-generator(n) that takes a parameter
2. Initialize counter:
   - set value to 0.
3. Generator values:
   - while value is less that 0:
   - Yield the current value.
   - Increment value by 1
4. Create Generator object:
   - call my-generator(11) to create a generator object
5. Increcate and print values:
   - For each value produced by the generator object:
   - Print value.

Program:

```
def my-generator(n):
    value = 0
    while value<n:
        yield value
        value += 1
for value in my-generator(3):
    print(value).
```

Output:

0
1
2.



Date: 17/9/25

6.2 Imagine you are working on a messaging application that needs to format messages differently based on the user's preferences users can change choose to have their messages autometically convert-ted to uppercase (for emphasis) or to lowercase (for a softer tone). You are provided with two decorators: upper case-decorator and lower case-decorator. These decorators modify the behaviour of the function they decorate by converting the text to uppercase or lowercase, respectively. write a program to implemented it.

Algorithm:

1. Create decorators.
   - Define upper case-decorator. to convert the result of a function to uppercase.
   - Define lowercase-decorator to convert the result of a function to lower case.
2. Define functions:
   - Define shout-function to return the input text. Apply
   - @ upper case-decorator to this function.
   - Define wishper function to return the input text. Apply @ lower case-decorator to this function.
3. Define great function:
   - Define great function that:
   - Accepts a function (func) as input
   - Calls this function with the text "Hi, I am created by a function passed as an argument".
   - prints the result.
4. Execute the program:
   - Call great (shout) to print the greating in uppercase.
   - Call great (whisper) to print the greating in lowercase.

Output:

HI, I AM CREATED BY A FUNCTION PASSED AS AN ARGUMENT.

hi, i am created by a function passed as an argument.

Program:

```
def uppercase_decorator(func):
    def wrapper(text):
        return func(text).upper()
    return wrapper.

def lowercase_decorator(func):
    def wrapper(text):
        return func(text).lower()
    return wrapper.

@uppercase_decorator.
def shout(text):
    return text.

@lowercase_decorator
def whisper(text):
    return text.

def greet(func):
    greeting = func("Hi, I am created by a function passed as an
                    argument.")
    print(greeting)

greet(shout)
greet(whisper).
```

VEL TECH	
EX NO.	8
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	
TOTAL (20)	15

Result: Thus, the python program to implement python generator and decorator was successfully executed and the output was verified.